MODULE 06 · DOMAIN 1 - ARCHITECTURE BUILT FOR THE CLOUD

AI Integration: Bedrock and Agents

Calling models across a multi-account estate, and giving agents tools without giving away the keys.

DOMAIN 1 of 5 EXAM WEIGHT 25% MODULE 6 of 52

admin@sthithapragna.com

Integrating generative AI is an **architecture** problem: access, safeguards, capacity, residency and cost.

The Bedrock pieces an architect works with

PIECE	WHAT IT IS FOR
Converse API	One request format across models, including tool use
Inference profiles	Cross-Region routing, and per-application cost tracking
Service tiers	Priority, Standard or Flex, per request
Guardrails	Content filters and safeguards on inputs and outputs
Knowledge Bases	Managed retrieval for RAG, covered in module 7
Agents and AgentCore	Models that call tools to complete tasks

ON THE EXAM

Only Bedrock and SageMaker AI appear in the guide's AI category. Amazon Q, Kendra and Comprehend do not.

Ways to pay for inference

OPTION	CHOOSE IT FOR
On-demand, Standard tier	Everyday interactive traffic
Priority tier	Latency-critical, user-facing; premium price
Flex tier	Non-interactive work that can wait; discounted
Batch inference	Large offline jobs submitted as files
Provisioned Throughput	Guaranteed capacity for one model
Cross-Region inference	More throughput by spreading across Regions

ON THE EXAM

Service tiers arrived in November 2025. Flex is the cost answer for agent steps and evaluations that can wait.

Geographic or global cross-Region inference?

GEOGRAPHIC PROFILE

Stays in a geography

- Routes within US, EU or APAC
- Standard pricing
- Suits data residency rules
- SCPs must allow each listed Region

GLOBAL PROFILE

Any commercial Region

- Highest available throughput
- About 10 percent cheaper
- No geographic boundary
- SCPs must allow RequestedRegion unspecified

THE TIEBREAKER

A residency requirement means geographic. No residency constraint and a cost goal means global.

A multi-account pattern for generative AI

01
Define
Responsible AI team builds a guardrail in the management account.

02
Enforce
A Bedrock policy in Organizations applies it to OUs.

03
Restrict
SCPs limit approved models and Regions.

04
Invoke
Workload accounts call Bedrock through VPC endpoints.

05
Observe
Invocation logs flow to the central log archive.

ON THE EXAM

Bedrock policies, since April 2026, enforce a guardrail on every invocation across an OU with no per-account setup.

Central gateway or governed direct access?

CENTRAL GATEWAY ACCOUNT

One front door

- Workloads call an internal API
- Central quotas, caching and model routing
- One place to rotate and govern access
- A shared bottleneck to scale

DIRECT, GOVERNED ACCESS

Each account calls Bedrock

- Bedrock policies enforce guardrails
- SCPs limit models and Regions
- Cost by IAM principal tag or app profile
- No extra service in the request path

THE TIEBREAKER

Custom routing, caching or central quotas justify a gateway. Consistent safeguards alone no longer do.

Controls that belong in every account

- **Bedrock policy** pins a guardrail version from the management account. One per Region.
- **SCP deny** on bedrock:InvokeModel for model ARNs that are not approved.
- **Interface VPC endpoints** for Bedrock runtime, with endpoint policies.
- **Model invocation logging** to CloudWatch Logs or S3. Off until enabled.
- **Prompts and outputs** are not used to train the models or shared with providers.

> ON THE EXAM
>
> A Bedrock policy is Regional. Enforcing in three Regions means a guardrail and policy for each.

An agent is a model in a loop: it chooses a tool, calls it, reads the result, and decides again.

Amazon Bedrock AgentCore components

COMPONENT	WHAT IT PROVIDES
Runtime	Isolated session per user; microVM sessions up to 8 hours
Gateway	Turns APIs, Lambda and OpenAPI specs into MCP tools
Memory	Short-term within a session, long-term across sessions
Identity	Inbound sign-in, outbound token vault for OAuth and API keys
Policy	Checks every tool call; deny by default, written in Cedar
Observability	OpenTelemetry traces, tokens and latency in CloudWatch

ON THE EXAM

AgentCore is framework- and model-agnostic. Code Interpreter, Browser and Evaluations complete the set.

Bedrock Agents or AgentCore?

BEDROCK AGENTS

Configure, do not code

- Instructions, action groups, knowledge bases
- Bedrock-hosted models
- Least code to a working agent
- Still supported for existing users

AGENTCORE

Bring your own agent

- Any framework: Strands, LangGraph, CrewAI
- Any model, inside or outside Bedrock
- MCP tools, VPC access, long sessions
- Agent-to-agent communication

THE TIEBREAKER

An existing framework, a non-Bedrock model, MCP or long sessions means AgentCore.

Giving agents tools safely

- **Least privilege per tool.** The agent's role should reach only what its tools need.
- **Credentials from Identity's token vault,** never in prompts, code or environment variables.
- **Policy intercepts each tool call,** so a confused model cannot exceed its limits.
- **Prompt injection** in retrieved content can steer an agent. Validate before acting.
- **High-impact actions** wait for human approval. Module 24 covers the pattern.

> ON THE EXAM
>
> Treat the model as untrusted. Enforce limits in the tool layer, where a prompt cannot argue its way past them.

Reading the scenario

WHEN THE QUESTION SAYS	IT IS TESTING
Swap models without code changes	Converse API
Throttled, but data must stay in the EU	Geographic cross-Region profile
Guardrail in every account, automatically	Bedrock policy in Organizations
Bedrock cost per team, no code changes	Tags on each team's IAM role
Existing LangGraph agent, hours-long sessions	AgentCore Runtime
Act on a user's behalf in a SaaS app	AgentCore Identity token vault

ON THE EXAM

Most AI questions are governance and capacity questions wearing an AI costume.

> Meridian Freight's support assistant currently calls one model through InvokeModel with a model-specific request body. The team wants to compare models from three providers and switch between them freely.
>
> Changing the model should not require changes to application code.

What should the solutions architect recommend?

A. Write an adapter for each provider's request format behind InvokeModel

B. Purchase Provisioned Throughput for each candidate model

C. Use the Converse API and read the model ID from configuration

D. Host each model on its own SageMaker AI endpoint

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Adapters per provider

Works, but it is code the Converse API already removes.

B Provisioned Throughput

Buys capacity. Does nothing for request format, and is costly for experiments.

C Converse API, model ID in config

One message format across providers. Switching is a configuration change.

D SageMaker AI endpoints

Adds hosting to manage and still varies per model.

THE SIGNAL

Model portability is what the Converse API exists for. The request body is the thing that changes per model.

> Meridian Freight's customer assistant runs in eu-west-1 and is throttled during the European morning peak. Contracts require that customer prompts are processed only in EU Regions.
>
> The company wants to remove the throttling as cost-effectively as possible.

What should the solutions architect do?

A Switch to the global cross-Region inference profile

B Switch to the EU geographic cross-Region inference profile

C Purchase Provisioned Throughput in eu-west-1 sized for the peak

D Send overflow requests to us-east-1 during the peak

◆ PAUSE AND DECIDE

What should the solutions architect do?

A Global profile

Cheapest and most capacity, but can process prompts outside the EU.

B EU geographic profile

Spreads load across EU Regions at standard pricing. Residency kept.

C Provisioned Throughput

Solves throttling, but you pay for peak capacity all day.

D Overflow to us-east-1

Breaks the EU processing requirement outright.

THE SIGNAL

Residency removes the global profile. Cost removes Provisioned Throughput. Geographic profiles sit between them.

> Meridian Freight has 60 AWS accounts, and teams in many of them are building generative AI features on Bedrock. The responsible AI team has defined a guardrail with content filters and denied topics.
>
> Every model invocation in every account must pass through that guardrail, even if a team forgets to configure it, with the LEAST operational overhead.

What should the solutions architect implement?

A. An AWS Config rule that flags applications calling Bedrock without a guardrail

B. A central gateway account that all Bedrock requests must pass through

C. An SCP that denies InvokeModel unless a guardrail identifier is supplied

D. A Bedrock policy in AWS Organizations, referencing the guardrail version, attached to the root

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Config rule

Detects after the fact. Unprotected invocations still happen.

B Central gateway

Can enforce it, but every request now depends on a service you build and run.

C SCP requiring a guardrail ID

Forces teams to pass one, but each must still configure it, and it breaks callers who forget.

D Bedrock policy at the root

Applies the guardrail to every invocation automatically, with no per-account setup.

THE SIGNAL

Even if a team forgets, with least overhead. Since April 2026 that is exactly what a Bedrock policy does.

> Twelve Meridian Freight product teams share one AI platform account. Each team's application calls Bedrock using its own IAM role.
>
> Finance wants monthly Bedrock cost by team in Cost Explorer. The platform team wants the LEAST effort, no change to the account structure, and no change to the teams' application code.

What should the solutions architect implement?

A. Tag each team's IAM role with a team tag and activate it as a cost allocation tag

B. A separate AWS account for each team

C. Parse invocation logs and multiply tokens by published prices

D. An application inference profile per team, tagged with a cost allocation tag

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Tagged IAM roles

Since April 2026, Bedrock cost is allocated by the calling principal's tags. No code changes.

B Account per team

Would work, but the scenario rules out changing the account structure.

C Parse logs

Builds a billing system by hand, and drifts from actual charges.

D Tagged application profiles

Also splits cost, but every team must change the model ID its code invokes.

THE SIGNAL

Separate IAM roles per team, and no code changes. Since April 2026, principal tags carry Bedrock cost allocation.

> Meridian Freight has built a claims-handling agent with LangGraph. Some claims take up to four hours of back-and-forth, and the agent must call internal APIs that are only reachable inside a VPC.
>
> The team wants each user's session isolated and does not want to rewrite the agent.

Where should the agent run?

A Rebuild it as a Bedrock Agent with action groups

B AgentCore Runtime, with AgentCore Gateway exposing the internal APIs as tools

C An AWS Lambda function invoked for each claim

D A self-managed ECS service with custom per-user session isolation

◆ PAUSE AND DECIDE

Where should the agent run?

A **Bedrock Agent**

Means rebuilding the agent in Bedrock Agents' configuration model. Ruled out.

B **AgentCore Runtime and Gateway**

Runs the existing LangGraph code, isolates each session, reaches the VPC.

C **Lambda**

A 15-minute limit cannot hold a four-hour session.

D **Self-managed ECS**

Possible, but you build session isolation yourself. More to run.

THE SIGNAL

Existing framework rules out Bedrock Agents. Four hours rules out Lambda. Isolation built in rules out DIY.

A Meridian Freight scheduling agent, running on AgentCore Runtime, books delivery slots in each customer's third-party calendar service. Customers grant access through that service's OAuth consent screen.

Security requires that tokens never appear in prompts, code or configuration files.

How should the agent obtain each customer's calendar credentials?

A Store each customer's refresh token in a Secrets Manager secret read by the agent code

B Include the access token in the system prompt for each session

C Give the agent an IAM role with permission to the calendar service

D Use AgentCore Identity, storing the OAuth tokens in its token vault for each user

◆ PAUSE AND DECIDE

How should the agent obtain each customer's calendar credentials?

A **Secrets Manager per customer**

Workable, but the agent code handles tokens and the OAuth flow itself.

B **Token in the prompt**

Breaks the requirement directly, and exposes the token to the model.

C **IAM role**

A third-party service does not accept AWS IAM credentials.

D **AgentCore Identity token vault**

Runs the OAuth consent flow and holds each user's tokens outside the agent.

THE SIGNAL

Delegated, per-user access to a third-party service is an outbound OAuth problem. AgentCore Identity is built for it.

What to carry out of this module

- **Converse API** for model portability. **Inference profiles** for routing and cost tags.
- **Geographic profile** when residency matters; **global** when it does not.
- **Bedrock policies** enforce a guardrail across an OU. Regional and version-pinned.
- **AgentCore** for existing frameworks, long sessions, MCP tools, delegated credentials.
- **Treat the model as untrusted.** Enforce limits in the tool layer.

> ON THE EXAM
>
> Every date in this module is 2025 or later. Older courses cannot have taught it.